ADVERSARIAL SEARCH AND GAMES

- Many decision-making problems involve multiple intelligent agents with conflicting objectives.
- Unlike classical search problems, where the environment is passive, game-playing problems require an agent to consider the actions of an opponent.
- The success of a move depends not only on the agent's decision but also on how other players respond.
- Games provide a convenient framework for studying strategic reasoning because their rules, states, and possible actions are usually well defined.
- Popular examples studied in Artificial Intelligence include Tic-Tac-Toe, Chess, Checkers, Go, etc.

Two-Player Zero-Sum Games

Two-player zero-sum games are games with two players who take turns and play against each other, and what is good for one player is equally bad for the other player.

Note: In a general two-player zero-sum game, the sum of the payoffs to both the players is constant for all possible outcomes of the game. More precisely, these are *constant-sum* games; however, the term "zero-sum games" is usually used, because it is always possible to redefine the payoffs such that the sum is zero, without changing the strategic structure of the game.

Examples: Tic-Tac-Toe, Chess, etc.

Formal Definition of a Game

A game can be defined with the following components:

- Set of states S, Set of players P, Set of actions A.
  For a two-player zero-sum game, P is usually taken as $\{\text{MAX}, \text{MIN}\}$, where MAX is the player who plays first.
- s_0 : The initial state, which specifies how the game is set up in the beginning.
- To-Move(s): A function from the set of states S to the set of players P – the player whose turn it is to move in state s.
- Actions(s): The set of legal moves in state s – a subset of A, the set of all actions.
- Result(s, a): A function from $S \times A$ to S – the transition model, which defines the state resulting from taking action a in state s.
- Is-Terminal(s): A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called terminal states.
- Utility(s, p): A function from $S \times P$ to $\mathbb{R}$ – a utility function (also called an objective function or payoff function), which defines the final numeric value to player p when the game ends in terminal state s.

Optimal Decisions in Two-Player Zero-Sum Games

Given a game, the optimal strategy can be determined using the "minimax value" of each state s, denoted by $\text{Minimax}(s)$.

$\text{Minimax}(s)$ is the utility of being in state s for MAX, assuming that both players play optimally from there to the end of the game.

For a terminal state s, $\text{Minimax}(s) = \text{Utility}(s, \text{MAX})$.

For a non-terminal state:
MAX prefers to move to a state of maximum value (for MAX) when it is MAX's turn to move.
MIN prefers to move to a state of minimum value (for MAX) when it is MIN's turn to move.

(*Note:* Minimum value for MAX is equivalent to maximum value for MIN.)

Therefore, $\text{Minimax}(s)$ is defined as follows:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s, \text{MAX}), & \text{if } s \text{ is a terminal state} \\ \max\limits_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)), & \text{if To-Move}(s) = \text{MAX} \\ \min\limits_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)), & \text{if To-Move}(s) = \text{MIN} \end{cases}$$

At a state s, the best action is the action that leads to a state of:

- maximum Minimax value among all the successor states of s, if $\text{To-Move}(s) = \text{MAX}$.
- minimum Minimax value among all the successor states of s, if $\text{To-Move}(s) = \text{MIN}$.

Note that the computation of Minimax value of a state s starts at terminal states and goes all the way back to state s.

```
function MINIMAX-SEARCH(state) returns an action
        player ← TO-MOVE(state)
        value, move ← MAX-VALUE(state)
        return move

function MAX-VALUE(state) returns a (utility, move) pair
        if IS-TERMINAL(state) then return UTILITY(state, player), null
        v ← −∞
        for each a in ACTIONS(state) do
                v′, a′ ← MIN-VALUE(RESULT(state, a))
                if v′ > v then v, move ← v′, a
        return v, move

function MIN-VALUE(state) returns a (utility, move) pair
        if IS-TERMINAL(state) then return UTILITY(state, player), null
        v ← +∞
        for each a in ACTIONS(state) do
                v′, a′ ← MAX-VALUE(RESULT(state, a))
                if v′ < v then v, move ← v′, a
        return v, move
```

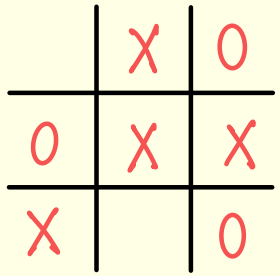
Example :

Tic-Tac-Toe with players MAX(X) and MIN(O):

For a terminal state s,

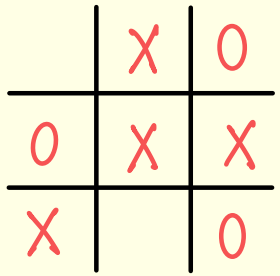
$$\text{Utility}(s, \text{MAX}) = \begin{cases} 1, & \text{if } X \text{ wins.} \\ -1, & \text{if } O \text{ wins.} \\ 0, & \text{if it is a draw.} \end{cases}$$

1) Find the minimax value of the following state:

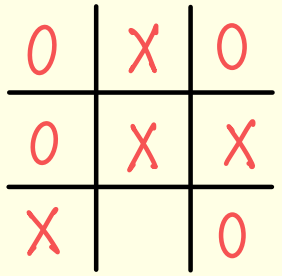
$s =$ 

; O's turn next.

(To-Move(s) = O.)

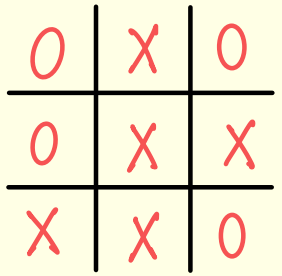


O → a



Minimax = 1

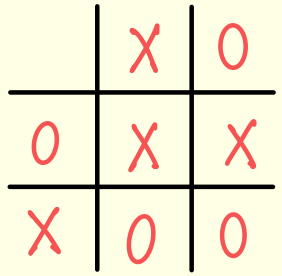
X →



Terminal state.

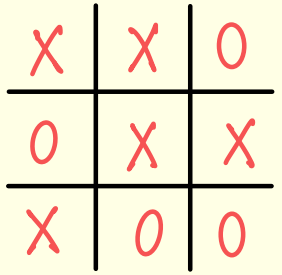
Minimax value = Utility = 1.

O → b



Minimax = 0

X →



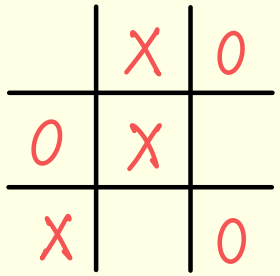
Terminal state.

Minimax value = Utility = 0.

Minimax = min{1, 0} = 0.

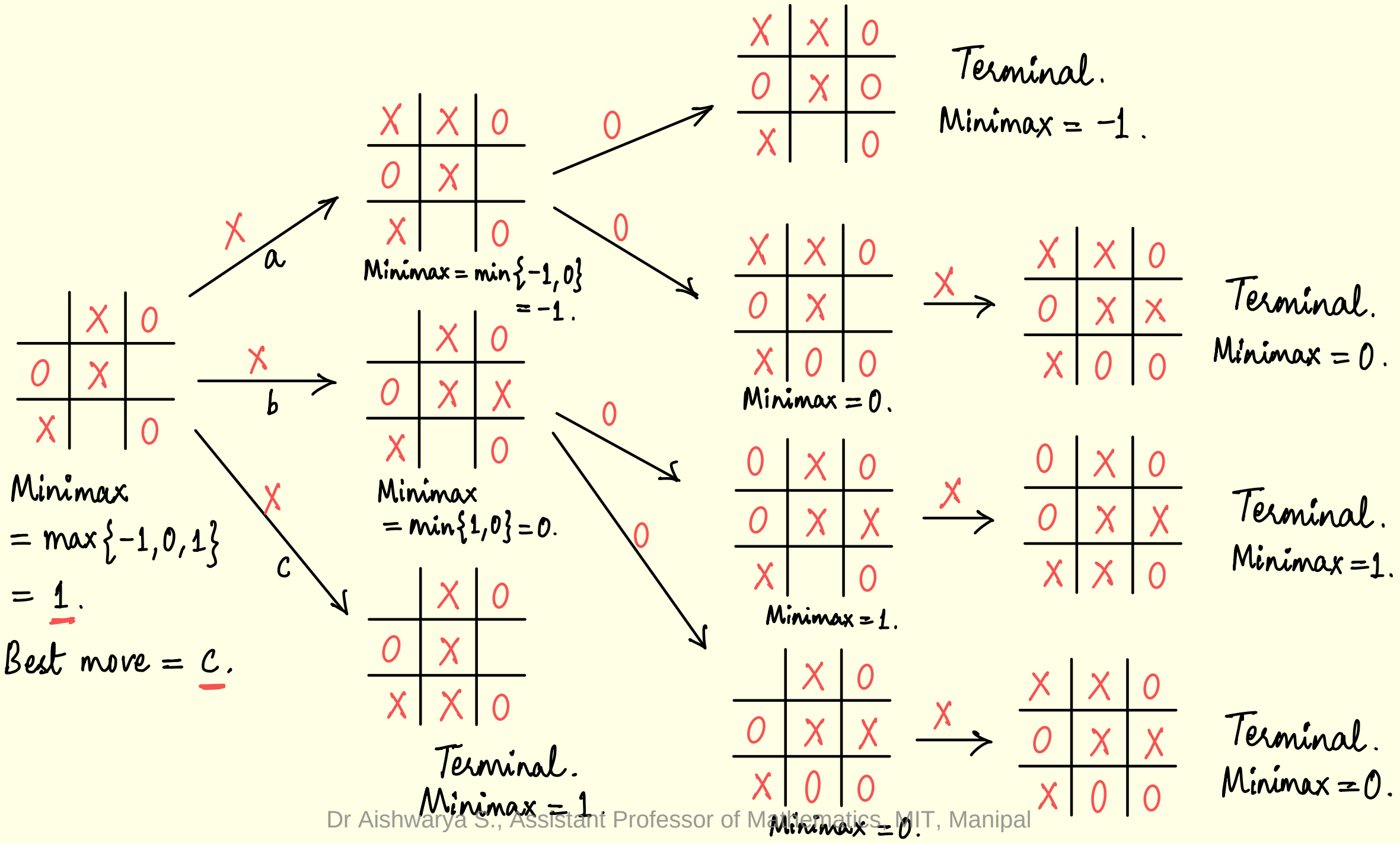
Best action = b.

2) Find the minimax value of the following state:

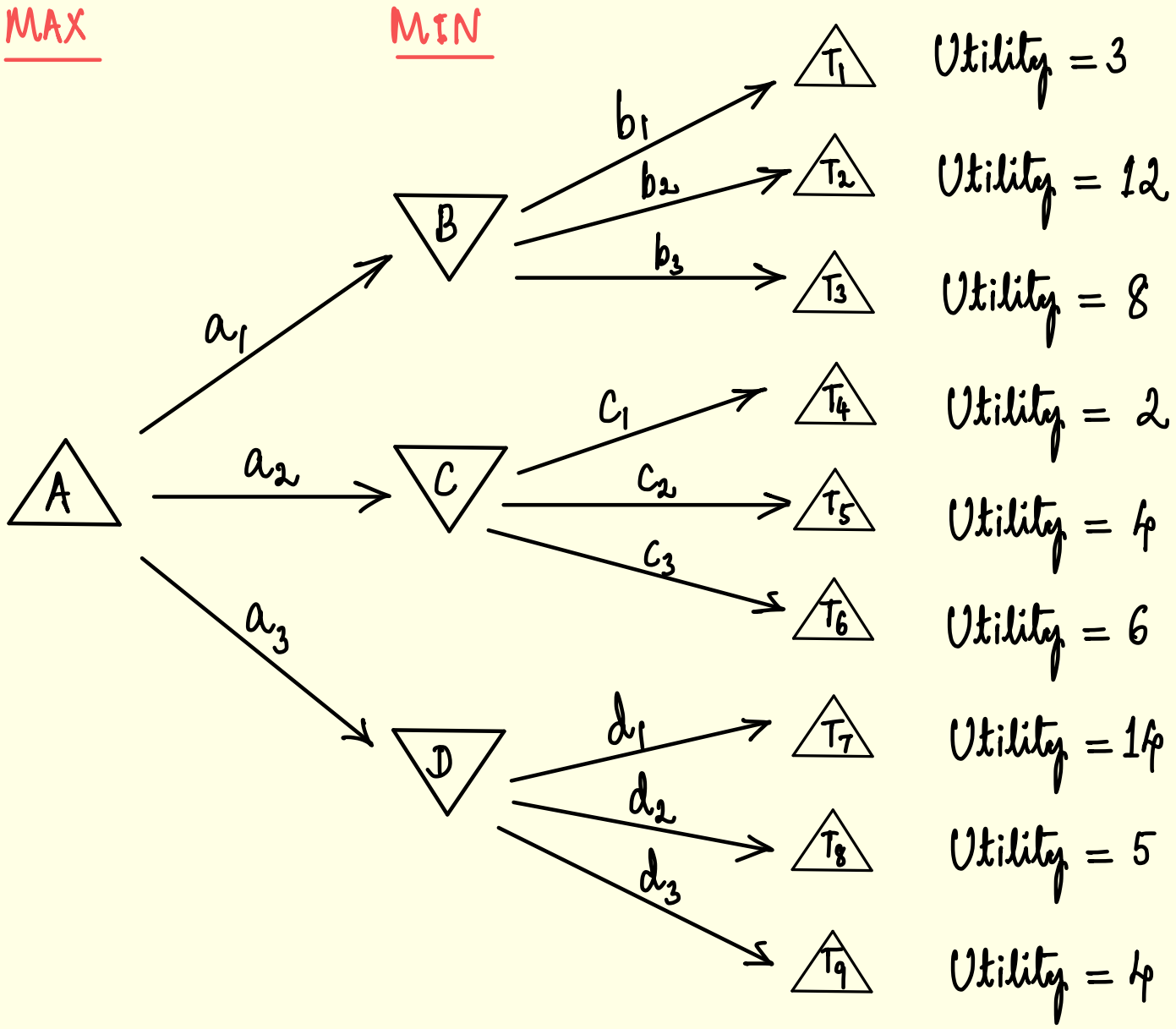
$s =$ 

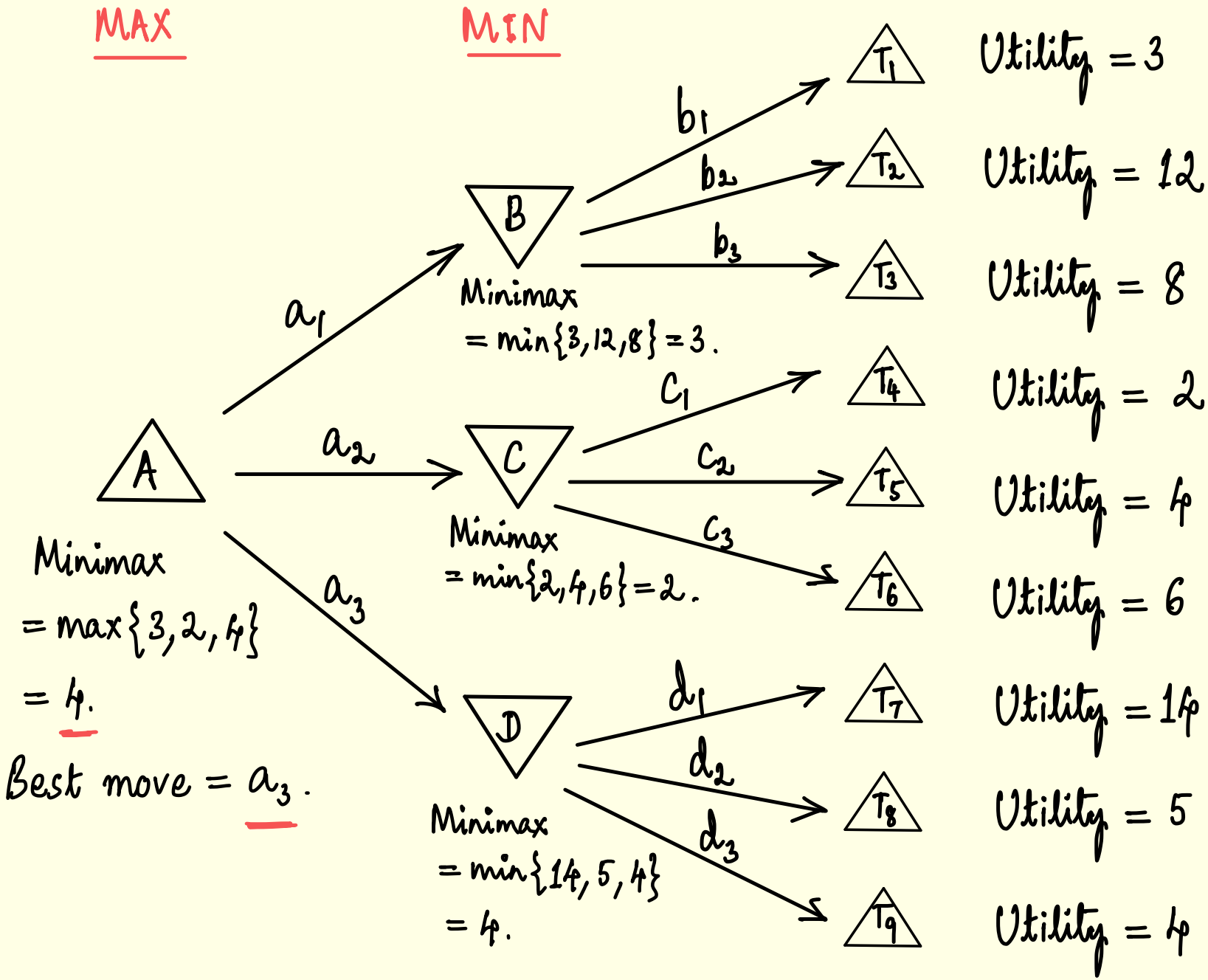
; X's turn next.

(To-Move(s) = X.)


Minimax = max{-1, 0, 1} = 1.
Best move = c.
a
b
c
Minimax = min{-1, 0} = -1.
Minimax = min{1, 0} = 0.
Terminal. Minimax = 1.
Terminal. Minimax = -1.
Minimax = 0.
Terminal. Minimax = 0.
Minimax = 1.
Terminal. Minimax = 1.
Minimax = 0.
Terminal. Minimax = 0.

3) Find the minimax value of state A given the following game tree:




MAX
MIN
A
B
C
D
a_1
a_2
a_3
b_1
b_2
b_3
c_1
c_2
c_3
d_1
d_2
d_3
T_1
T_2
T_3
T_4
T_5
T_6
T_7
T_8
T_9
Utility = 3
Utility = 12
Utility = 8
Utility = 2
Utility = 4
Utility = 6
Utility = 14
Utility = 5
Utility = 4
Minimax = min{3, 12, 8} = 3.
Minimax = min{2, 4, 6} = 2.
Minimax = min{14, 5, 4} = 4.
Minimax = max{3, 2, 4} = 4.
Best move = a_3.

The Minimax Search algorithm performs a depth-first search of the game tree, starting from the node corresponding to the input state.

If there are b legal moves from each node and the maximum depth of the tree is m, then the time complexity of the minimax-search algorithm is $O(b^m)$.

The exponential time complexity makes minimax-search practically infeasible for complex games.

For example, the game tree for chess has an average branching factor of about 35 and an average depth of about 80.
Therefore, the total number of states in an average game tree is about $35^{80} \approx 10^{123}$ and it is not practically feasible to search these states.

Can we find the minimax values in a cleverer way?

Yes - by skipping the nodes that are irrelevant to the minimax value of the input state.

This is known as ***alpha-beta pruning*** of the game tree.

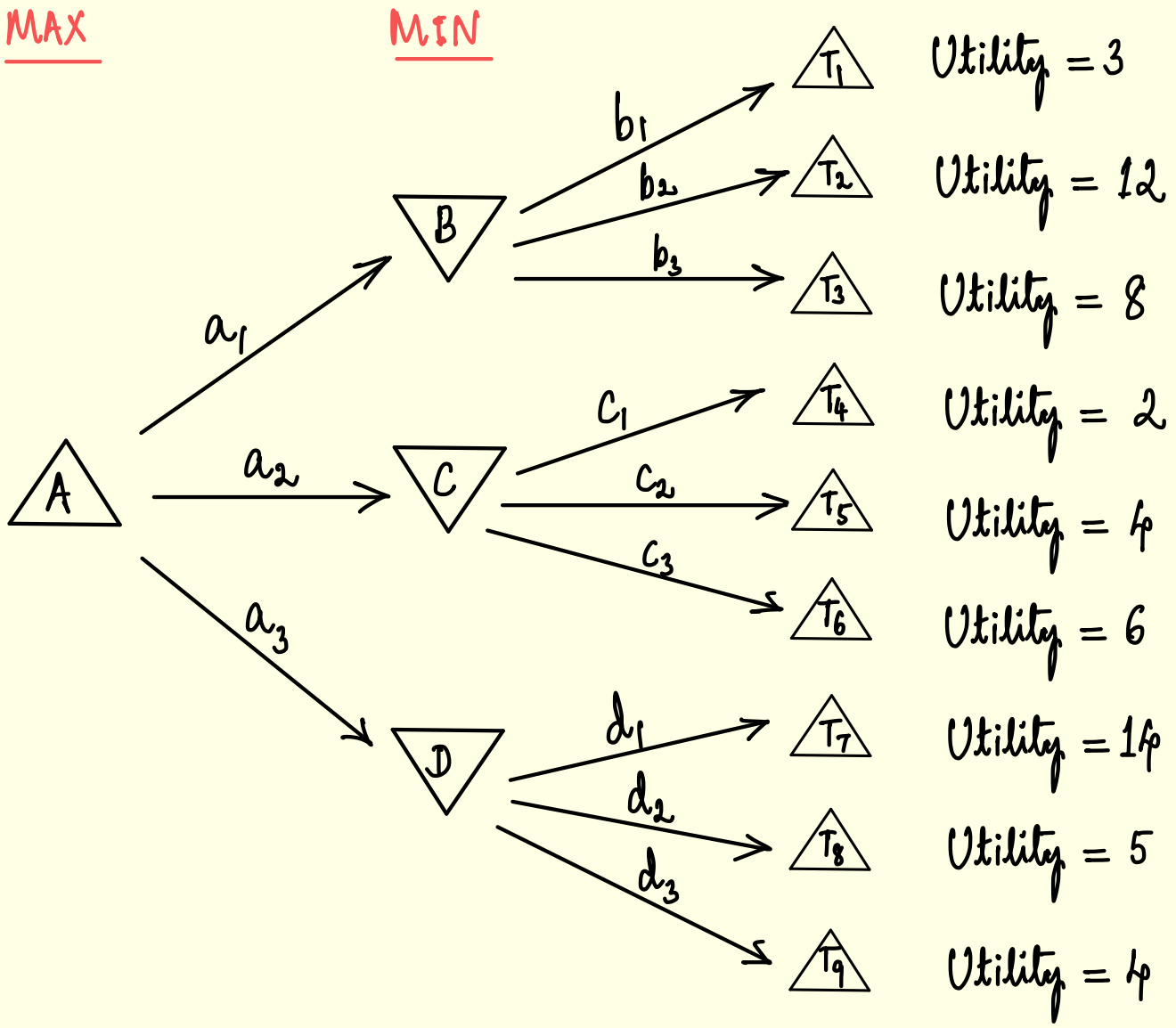
The **ALPHA-BETA-SEARCH** algorithm is the improvement over MINIMAX-SEARCH algorithm obtained by incorporating alpha-beta pruning.

function ALPHA-BETA-SEARCH(*state*) **returns** *an action*
    player ← TO-MOVE(*state*)
    value, *move* ← MAX-VALUE(*state*, $-\infty$, $+\infty$)
    return *move*

function MAX-VALUE(*state*, α, β) **returns** a (*utility*, *move*) pair
    if IS-TERMINAL(*state*) **then return** UTILITY(*state*, *player*), *null*
    $v \leftarrow -\infty$
    for each *a* **in** ACTIONS(*state*) do
        $v', a' \leftarrow$ MIN-VALUE(RESULT(*state*, *a*), α, β)
        if $v' > v$ **then**
            v, *move* ← v', *a*
            $\alpha \leftarrow \max\{\alpha, v\}$
        if $v \geq \beta$ **then return** *v*, *move*
    return *v*, *move*

function MIN-VALUE(*state*, α, β) **returns** a (*utility*, *move*) pair
    if IS-TERMINAL(**state**) **then return** UTILITY(*state*, *player*), *null*
    $v \leftarrow +\infty$
    for each *a* **in** ACTIONS(*state*) do
        $v', a' \leftarrow$ MAX-VALUE(RESULT(*state*, *a*), α, β)
        if $v' < v$ **then**
            v, *move* ← v', *a*
            $\beta \leftarrow \min\{\beta, v\}$
        if $v \leq \alpha$ **then return** *v*, *move*
    return *v*, *move*

Example :


MAX
MIN
A
B
C
D
a_1
a_2
a_3
b_1
b_2
b_3
c_1
c_2
c_3
d_1
d_2
d_3
T_1
T_2
T_3
T_4
T_5
T_6
T_7
T_8
T_9
Utility = 3
Utility = 12
Utility = 8
Utility = 2
Utility = 4
Utility = 6
Utility = 14
Utility = 5
Utility = 4

ALPHA-BETA-SEARCH(A):
player ← MAX

#	Level	Function call	Inherited (α, β)	Result	Note
1	0	MAX-VALUE$(A, -\infty, +\infty)$	$(-\infty, +\infty)$		
2	1	MIN-VALUE$(B, -\infty, +\infty)$	$(-\infty, +\infty)$		
3	2	MAX-VALUE$(T_1, -\infty, +\infty)$	$(-\infty, +\infty)$	**3, *null***	$\beta = min\{+\infty, 3\} = 3$
4	2	MAX-VALUE$(T_2, -\infty, 3)$	$(-\infty, 3)$	**12, *null***	No update
5	2	MAX-VALUE$(T_3, -\infty, 3)$	$(-\infty, 3)$	**8, *null***	No update
-	1	MIN-VALUE$(B, -\infty, +\infty)$ returns		**3, b_1**	$\alpha = max\{-\infty, 3\} = 3$
6	1	MIN-VALUE$(C, 3, +\infty)$	$(3, +\infty)$		
7	2	MAX-VALUE$(T_4, 3, +\infty)$	$(3, +\infty)$	**2, *null***	$\beta = min\{+\infty, 2\} = 2$ $v = 2 \leq \alpha(= 3) \rightarrow$ **Prune remaining child nodes**
-	1	MIN-VALUE$(C, 3, +\infty)$ returns		**2, c_1**	No update

#	Level	Function call	Inherited (α, β)	Result	Note
8	1	MIN-VALUE$(D, 3, +\infty)$	$(3, +\infty)$		
9	2	MAX-VALUE$(T_7, 3, +\infty)$	$(3, +\infty)$	**14, *null***	$\beta = min\{+\infty, 14\} = 14$
10	2	MAX-VALUE$(T_8, 3, 14)$	$(3, 14)$	**5, *null***	$\beta = min\{14, 5\} = 5$
11	2	MAX-VALUE$(T_9, 3, 5)$	$(3, 5)$	**4, *null***	$\beta = min\{5, 4\} = 4$
-	1	MIN-VALUE$(D, 3, +\infty)$ returns		**4, d_3**	$\alpha = max\{-\infty, 4\} = 4$
-	0	MAX-VALUE$(A, -\infty, +\infty)$ returns		**4, a_3**	

ALPHA-BETA-SEARCH(A) returns $\boldsymbol{a_3}$ as the best move at state A.

Note:

In (plain) MINIMAX-SEARCH algorithm, all the intermediate calls of MAX-VALUE and MIN-VALUE functions return the true minimax values of the corresponding subtrees.

However, in ALPHA-BETA-SEARCH algorithm, the values returned by the intermediate calls of MAX-VALUE and MIN-VALUE functions are not necessarily the true minimax values of the corresponding subtrees. The value it returns is only a **bound** on the true minimax value of that subtree, that is provably good enough for the decision at the parent state.

The ALPHA-BETA-SEARCH algorithm guarantees to return the best move only for the root node (that is, the input state given to it). This output is identical to what plain minimax-search would return for the same input state. Thus, the point of alpha-beta-search is to get the same answer as minimax-search, but with less work.

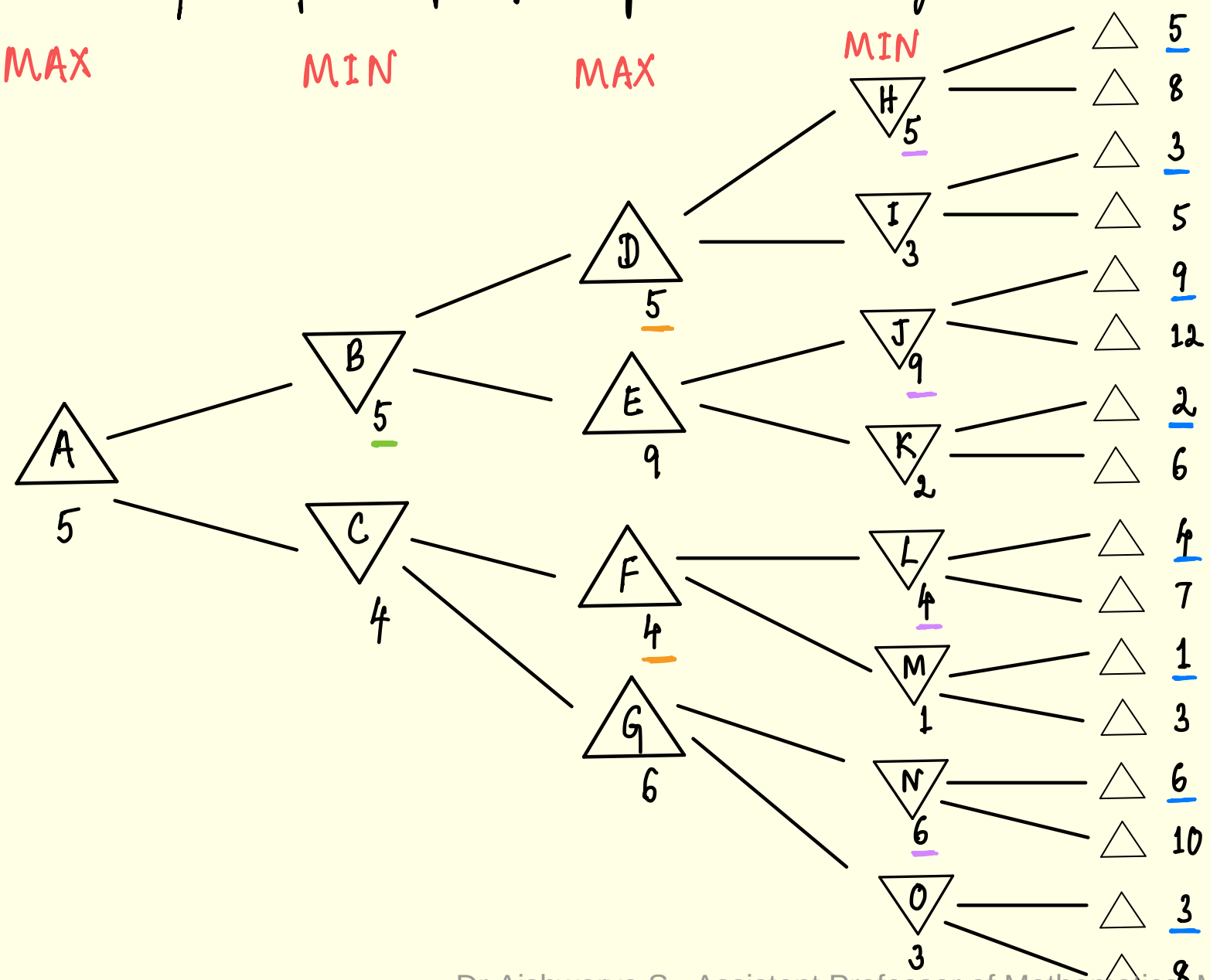
When does alpha-beta-search give the maximum improvement?

Clearly, the effectiveness of alpha-beta pruning is highly dependent on the order in which the child states of any node are examined.

With the "perfect" ordering of moves at every state, alpha-beta-search needs only $O(b^{m/2})$ nodes, instead of $O(b^m)$ of minimax-search. This means that the effective branching factor reduces from b to $\sqrt{b}$.

For chess, this means a branching factor of about 6 (instead of 35).

An example of a perfectly ordered game tree:



Even an alpha-beta-search with better move ordering is not feasible for games like chess; because there are still too many states to explore in the available time.

In the first paper on computer game-playing, *Programming a Computer for Playing Chess* (Shannon, 1950*), Claude Shannon recognized this problem and proposed two strategies:

Type A strategy, which considers all possible moves to a certain depth in the search tree and then uses a heuristic function to estimate the utility of states at that depth. It explores a wide but shallow portion of the tree.

Type B strategy, which ignores moves that look bad, and follows promising paths "as far as possible". It explores a deep but narrow portion of the tree.

***Reference:** C. E. Shannon, *Programming a Computer for Playing Chess,* Philosophical Magazine, Volume 41 (7th series), 256-275 (1950).

Some key ideas used in practice (heuristic alpha-beta search):

- **Cutoff test replacing terminal test:** Stop at a depth limit instead of searching till the end of the game.
- **Evaluation function replacing utility function:** Estimate how good a non-terminal position is (for example, using king safety, piece mobility, etc. in chess) instead of computing the true game value.
- **Move ordering:** Searching the most promising moves first (for example, using prior knowledge or iterative deepening results) makes alpha-beta pruning far more effective, potentially reducing effective branching factor dramatically.
- **Transposition tables:** Caching previously seen positions (since different move orders can reach the same position) avoids redundant search.
- **Forward pruning / selective search:** Deliberately ignoring moves that look bad, at the risk of missing something (used carefully, since it is not guaranteed safe).
- **Iterative deepening:** Run depth-limited search with increasing depth, useful for time management and improving move ordering.

Beyond alpha-beta-search, the following techniques are also used:

- **Monte Carlo Tree Search (MCTS):** Used in modern game AI (Example: Go, and hybrid approaches in chess engines) - builds a search tree using random/guided playouts rather than exhaustive expansion.

- **Learned evaluation functions:** Systems such as AlphaZero use deep neural networks (trained through self-play/reinforcement learning) to learn an evaluation function, along with Monte Carlo Tree Search to evaluate board positions and select the most promising moves.